

To allow the user to temporarily go to an elevated level and use root privileges, we will add the user to the *sudo* group using the following command:

```
usermod -aG sudo webadmin
```

Now, you can use *sudo* to call commands as a super user.

Setting up a basic firewall: On a public network like the Internet, we cannot keep our gates open everywhere to allow everyone in. As a rule of thumb, restrict access to everything in the system, and allow access to a specific audience only whenever needed.

Let's set up a basic firewall known as Uncomplicated Fire Wall (UFW).

To see the list of applications registered in UFW, use the command below:

```
sudo ufw app list
```

Deny all incoming connections by default, as per policy:

```
sudo ufw default deny incoming
```

To make sure SSH is allowed through UFW for us to connect, we allow OpenSSH through UFW, as follows:

```
sudo ufw allow OpenSSH
```

SSH access: Since we are going to control the remote server via SSH, we need a way to authenticate ourselves to log in to the system.

One way is by using passwords. Password authentication is susceptible to brute-force attacks and is generally discouraged. The safer way is to set up SSH keys to communicate with the remote server and the local machine.

This process involves generating an RSA keypair using the command *ssh-keygen* and passing the public key to the server (generally using *ssh-copy-id*). Once the key has been copied, you can go ahead and disable password authentication for SSH by editing the */etc/ssh/sshd_config* file and uncommenting/adding the following directive:

```
PasswordAuthentication no
```

This ensures that you can log in to the server via SSH only using the generated keys and not the weaker password authentication system.



Warning: Make sure you are able to log in using key-based authentication on the non-root account with *sudo* permissions before disabling password authentication.

If your server supports SSH through only a specific IP range (like Amazon AWS), you can use it to set a range

```
dedipyaman@2D: ~
dedipyaman@2D:~$ sudo ufw app list
Available applications:
Apache
Apache Full
Apache Secure
Bind9
CUPS
OpenSSH
Samba
dedipyaman@2D:~$
```

Figure 1: UFW app list

of specific IP addresses to be able to access the machine remotely using SSH.

Securing Apache

First, allow Apache in UFW, as follows:

```
sudo ufw allow in "Apache Full"
```

Disable open indexes: Open indexes can expose your directory structure which may prove to be fatal. You can disable open indexes by editing */etc/apache2/apache2.conf*:

```
<Directory /var/www/html>
    Options -Indexes
</Directory>
```

Or just add *Options -Indexes* in the *.htaccess* file in the root directory.

Install *mod_evasive* and *mod_security*: *mod_security* acts as a Web application firewall (WAF) to tighten app security, and *mod_evasive* helps protect it from potential attacks like brute force and DDoS. To install these, give the following command:

```
sudo apt-get install libapache2-mod-security2 libapache2-mod-evasive
```

While the standard installation should be enough for most people, you might have to configure it separately to set things according to your needs.

Deny access to certain directories: You might want to deny access to certain directories, in which case we can use the *Deny* from all directives for this in the *apache.conf* file or in *htaccess*.

```
<Directory ~ "\secret">
    Options None
    Order allow,deny
    Deny from all
</Directory>
```

Using SSL: Secure Socket Layer (SSL) can be used to encrypt connections between the user and the server. It allows your website to be accessible over port 443 (HTTPS). You need to obtain an SSL certificate from a vendor like Comodo